



STP 协议使用手册

版本	1.1.0
作者	固件开发组
部门	芯片安全事业部
日期	2025 - 09 - 03
分类	内部文档

Copyright Notice and Proprietary Information Notice

Copyright © 2014 - 2025 Open Security Research, Inc. All rights reserved. This documentation contains confidential and proprietary information that is the property of Open Security Research, Inc. The documentation is transported under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, translated, distributed, disclosed, announced or copied in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Open Security Research, Inc., or as expressly provided by the license agreement.

版权声明和专有信息声明

版权所有 © 2014 - 2025 深圳市纽创信安科技开发有限公司 保留所有权利。本文档包含机密和专有信息，属于深圳市纽创信安科技开发有限公司的财产。该文档根据对应的许可协议传播，且只能根据许可协议的条款使用或复制。未经深圳市纽创信安科技开发有限公司事先书面许可或经许可协议明确规定，该文档和对应软件的全部或任何部分均不得以任何形式或方式（电子的，机械的，手动的，光学的或其他的）被复制、传播、翻译、分发、披露、宣布或抄袭。

修订历史

版本	修改日期	描述
1.0.0	2025-08-22	初版
1.1.0	2025-09-03	• 增加服务端和客户端角色，区分默认状态 • 增加字符串帧功能

目录

1 协议简介	5
1.1 基本概念	5
2 环境要求	5
2.1 C 语言实现	5
2.2 Python 实现	5
3 用户适配指南	6
3.1 HAL 层实现 (C 语言)	6
3.1.1 必须实现的函数	6
3.1.2 可选实现的函数	6
3.1.3 返回值规范	6
3.2 基本使用流程	6
3.2.1 C 语言实现	6
3.2.2 Python 实现	7
4 测试方法	8
4.1 环境准备	8
4.1.1 硬件连接	8
4.1.2 C 语言测试	8
4.1.3 Python 测试	8
5 注意事项	8
5.1 重要提醒	8
5.2 常见问题	9
5.2.1 初始化失败	9
5.2.2 连接超时	9
5.2.3 数据传输错误	9
5.2.4 权限问题 (Linux)	9
5.3 性能建议	9
5.4 移植要点	9
5.4.1 嵌入式系统移植	9
5.4.2 跨平台注意事项	10

1 协议简介

STP (Safe Transport Protocol) 是一个基于 UART 的可靠传输协议，专为串行通信设计。该协议具有以下特点：

- **半双工通信**：支持发送/接收两种工作模式
- **帧结构化**：采用标准帧格式，包含起始字节、类型、负载长度、数据和 CRC 校验
- **可靠传输**：支持 CRC16 校验、自动重传和错误恢复机制
- **任意数据长度**：自动分块处理大数据，对用户透明
- **跨平台支持**：提供 C 语言和 Python 两种实现

1.1 基本概念

协议区分客户端和服务端两种角色：

- **客户端 (is_client=true)**：主动发起方，初始为发送模式，负责建立连接
- **服务端 (is_client=false)**：被动响应方，初始为接收模式，等待连接请求

协议帧类型：

- DATA (0x10)：数据帧，携带用户数据
- ACK (0x20)：确认帧，确认收到数据
- SWITCH (0x40)：模式切换帧，切换发送/接收模式
- RESYNC_REQ (0x08)：重新同步请求
- RESYNC_ACK (0x04)：重新同步响应
- ERR (0x80)：错误帧，指示传输错误
- STRING (0xFE 0xF0...0xFE 0xFE)：字符串帧，用于传输调试信息，不影响数据传输流程

2 环境要求

2.1 C 语言实现

- 编译器：GCC 4.8+ 或支持 C99 标准
- 操作系统：Windows/Linux/嵌入式
- UART 硬件：标准串口，支持配置波特率
- 内存：约 2KB（包含接收缓存和协议状态）

2.2 Python 实现

- Python：3.7+
- 依赖库：pyserial 3.0+
- 操作系统：Windows/Linux/macOS

安装 Python 依赖：

```
1| pip install pyserial
```

3 用户适配指南

3.1 HAL 层实现（C 语言）

C 语言实现需要用户实现硬件抽象层（HAL）接口：

3.1.1 必须实现的函数

```
1| // 初始化UART硬件，配置波特率等参数
2| uint32_t uart_init(void);
3|
4| // 发送单字节数据，阻塞直到发送完成
5| uint32_t uart_send_byte(uint8_t val);
6|
7| // 非阻塞读取单字节数据，无数据时返回STP_ERR_PEEK_NO_DATA
8| uint32_t uart_peek_byte(uint8_t* val);
9|
10| // 关闭UART硬件，释放资源
11| void uart_deinit(void);
```

3.1.2 可选实现的函数

```
1| // 获取毫秒时间戳，用于超时检测（使用弱符号，可选实现）
2| uint64_t stp_get_time_ms(void);
```

3.1.3 返回值规范

HAL 函数应返回以下错误码：

- STP_OK (0)：操作成功
- STP_ERR_HAL (6)：硬件操作失败
- STP_ERR_PEEK_NO_DATA (3)：uart_peek_byte 无数据可读

3.2 基本使用流程

3.2.1 C 语言实现

```
1| #include "strans_pro.h"
2|
3| // 字符串输出回调函数（可选）
4| void string_output_handler(const char *str, uint32_t len) {
```

```
5|     printf("收到调试信息: %.*s\n", (int)len, str);
6| }
7|
8| // 1. 初始化协议 (TRUE=客户端, FALSE=服务端, string_output_handler为字符串回调)
9| if (stp_init(TRUE, string_output_handler) != STP_OK) {
10|     // 处理初始化失败
11| }
12|
13| // 2. 发送数据
14| uint8_t data[] = "Hello STP!";
15| if (stp_send(data, sizeof(data)) != STP_OK) {
16|     // 处理发送失败
17| }
18|
19| // 3. 接收数据
20| uint8_t buffer[100];
21| if (stp_recv(buffer, sizeof(buffer)) != STP_OK) {
22|     // 处理接收失败
23| }
24|
25| // 4. 发送调试字符串 (可选)
26| uint8_t debug_msg[] = "Debug: operation completed";
27| stp_send_string(debug_msg, sizeof(debug_msg) - 1);
28|
29| // 5. 清理资源
30| stp_deinit();
```

3.2.2 Python 实现

```
1| from stp_uart import STPUart
2|
3| # 字符串输出回调函数 (可选)
4| def string_output_handler(data: bytes):
5|     print(f"收到调试信息: {data.decode('utf-8', errors='ignore')}")
6|
7| # 使用with语句自动管理资源
8| with STPUart(is_client=True, port="COM1", baudrate=57600,
9|             string_callback=string_output_handler) as stp:
10|     # 发送数据
11|     stp.send(b"Hello STP!")
12|
13|     # 接收数据
14|     response = stp.recv(100)
15|
16|     # 发送调试字符串 (可选)
17|     stp.send_string(b"Debug: operation completed")
```

4 测试方法

4.1 环境准备

4.1.1 硬件连接

- 需要两个串口端口（COM1/COM2 或 /dev/ttyUSB0/ttyUSB1）
- 使用串口线缆连接两个端口
- 或使用硬件环回进行单机测试

4.1.2 C 语言测试

```
1| cd tests
2| make
3| # 启动服务器
4| ./stp_server
5|
6| # 在另一终端启动客户端
7| python tests/client.py
```

4.1.3 Python 测试

```
1| # 启动服务器
2| python tests/server.py
3|
4| # 在另一终端启动客户端
5| python tests/client.py
```

另外可使用 `comprehensive_test.py` 脚本进行更多协议功能测试。

5 注意事项

5.1 重要提醒

1. **必须先实现 HAL 层**：C 语言版本需要用户实现 `uart_hal.h` 中定义的接口函数
2. **角色明确**：必须明确指定客户端和服务端角色，客户端负责发起连接
3. **资源管理**：使用完毕后必须调用 `stp_deinit()` 或 `close()` 释放资源
4. **错误处理**：建议检查所有函数返回值，特别是初始化和数据传输
5. **波特率一致**：通信双方必须使用相同的波特率配置
6. **字符串帧使用**：字符串帧用于调试信息，不影响正常数据传输，最大长度 1024 字节

5.2 常见问题

5.2.1 初始化失败

- 检查串口是否被其他程序占用
- 确认用户有串口设备访问权限（Linux 需要加入 dialout 组）
- 验证串口设备名称正确
- 检查 HAL 层实现是否正确

5.2.2 连接超时

- 确认对端程序已启动且处于正确模式
- 检查串口连线是否正确
- 验证波特率设置是否一致

5.2.3 数据传输错误

- 检查 CRC 校验是否正确
- 确认串口硬件连接稳定
- 验证 HAL 层发送和接收功能

5.2.4 权限问题（Linux）

```
1| # 将用户添加到dialout组
2| sudo usermod -a -G dialout $USER
3| # 重新登录或使用newgrp dialout使权限生效
```

5.3 性能建议

1. 波特率选择：推荐使用 57600 或 115200 波特率
2. 大数据传输：协议自动处理分块，用户无需手动分割
3. 错误恢复：连续错误时可调用 `stp_reset_comm()` 重置状态
4. 超时设置：默认字节超时时间适合大多数应用，特殊环境需要可调整
5. 字符串帧建议：用于调试输出，生产环境建议传入 NULL 禁用以提高性能

5.4 移植要点

5.4.1 嵌入式系统移植

1. 实现目标平台的 HAL 接口
2. 确保有足够内存（约 2KB）
3. 可集成到 UART 中断处理中提高响应性
4. 考虑实时性要求调整超时参数

5.4.2 跨平台注意事项

1. Windows: 串口名如 “COM1” , “COM2”
2. Linux: 串口设备如 “/dev/ttyUSB0” , “/dev/ttyACM0”
3. 注意不同平台的换行符和文件路径分隔符

通过遵循以上指南和注意事项，用户可以快速集成和使用 STP 协议库，实现可靠的串口通信功能。